

LeetCode SQL 50

Varshitha

"Data is the new currency, and SQL is the universal language that unlocks its true value."

TOPICS & SECTIONS (CLICK TO NAVIGATE)

- **Topic 1: Select** **5 Questions (Q1 – Q5)** →
- **Topic 2: Basic Joins** **9 Questions (Q1 – Q9)** →
- **Topic 3: Basic Aggregate Functions** **8 Questions (Q1 – Q8)** →
- **Topic 4: Sorting and Grouping** **7 Questions (Q1 – Q7)** →
- **Topic 5: Advanced Select & Joins** **7 Questions (Q1 – Q7)** →
- **Topic 6: Subqueries** **7 Questions (Q1 – Q7)** →
- **Topic 7: Advanced String Functions / Regex / Clause** **7 Questions (Q1 – Q7)** →

• Topic 1: Select

Fundamental row selection, boolean filtering, null handling, and string length logic.

Q1 Recyclable and Low Fat Products

EASY

Write a solution to find the IDs of products that are both low fat (`low_fats = 'Y'`) and recyclable (`recyclable = 'Y'`). Return the result table in any order.

♥ **KEY INSIGHT:** Simple compound boolean filtering via AND condition on indexed/enumerated columns.

INPUT: PRODUCTS

product_id	low_fats	recyclable
0	Y	N
1	Y	Y
2	N	Y
3	Y	Y
4	N	N

EXPECTED OUTPUT

product_id
1
3

```
SELECT product_id
FROM Products
WHERE low_fats = 'Y' AND recyclable = 'Y';
```

Q2 Find Customer Referee

EASY

Find the names of the customer that are not referred by the customer with id = 2. Return the result table in any order.

♥ **KEY INSIGHT:** In 3-valued SQL logic, `referee_id != 2` yields UNKNOWN for NULL values; explicitly add `OR referee_id IS NULL`.

INPUT: CUSTOMER

id	name	referee_id
1	Will	null
2	Jane	null
3	Alex	2
4	Bill	null
5	Zack	1
6	Mark	2

EXPECTED OUTPUT

name
Will
Jane
Bill
Zack

```
SELECT name
FROM Customer
WHERE referee_id != 2 OR referee_id IS NULL;
```

Q3 Big Countries

EASY

A country is big if it has an area of at least 3,000,000 km² or a population of at least 25,000,000. Write a solution to find the name, population, and area of the big countries.

♥ **KEY INSIGHT:** Standard disjunctive OR condition filtering multiple threshold attributes.

INPUT: WORLD

name	continent	area	population	gdp
Afghanistan	Asia	652230	25500100	20343000000
Albania	Europe	28748	2831741	12960000000
Algeria	Africa	2381741	37100000	188681000000
Andorra	Europe	468	78115	3712000000
Angola	Africa	1246700	20609294	100990000000

EXPECTED OUTPUT

name	population	area
Afghanistan	25500100	652230
Algeria	37100000	2381741

```
# Write your MySQL query statement below
SELECT name, population, area
FROM World
WHERE area >= 3000000 OR population >= 25000000;
```

Q4 Article Views I**EASY**

Write a solution to find all the authors that viewed at least one of their own articles. Return the result table sorted by id in ascending order.

♥ **KEY INSIGHT:** Self-comparison across row columns (`author_id = viewer_id`) paired with `DISTINCT` to eliminate duplicates.

INPUT: VIEWS

article_id	author_id	viewer_id	view_date
1	3	5	2019-08-01
1	3	6	2019-08-02
2	7	7	2019-08-01
2	7	6	2019-08-02
4	7	1	2019-07-22
3	4	4	2019-07-21
3	4	4	2019-07-21

EXPECTED OUTPUT

id
4
7

Write your MySQL query statement below

```
SELECT DISTINCT author_id AS id
FROM Views
WHERE author_id = viewer_id
ORDER BY author_id;
```

Q5 Invalid Tweets**EASY**

Write a solution to find the IDs of the invalid tweets. A tweet is invalid if the number of characters used in the content of the tweet is strictly greater than 15.

♥ **KEY INSIGHT:** Use `CHAR_LENGTH()` (character count) rather than `LENGTH()` (byte size) for multi-byte Unicode string safety.

INPUT: TWEETS

tweet_id	content
1	Vote for BI
2	Let us make America great again!

EXPECTED OUTPUT

tweet_id
2

Write your MySQL query statement below

```
SELECT tweet_id
FROM Tweets
WHERE LENGTH(content) > 15;
```

● Topic 2: Basic Joins

Multi-table associations, self-joins, cross product generations, and outer join null detections.

Q1 Replace Employee ID With The Unique Identifier

EASY

Write a solution to show the unique ID of each user. If a user does not have a unique ID replacement just show null.

♥ **KEY INSIGHT:** LEFT JOIN preserves all records from the primary entity table, filling missing foreign matches with NULL.

INPUT: EMPLOYEES

id	name
1	Alice
7	Bob
11	Meir
90	Winston
3	Jonathan

INPUT: EMPLOYEEUNI

id	unique_id
3	1
11	2
90	3

EXPECTED OUTPUT

unique_id	name
null	Alice
null	Bob
2	Meir
3	Winston
1	Jonathan

```
SELECT a.unique_id, b.name
FROM Employees b
LEFT JOIN EmployeeUNI a
  ON b.id = a.id;
```

Q2 Product Sales Analysis I

EASY

Write a solution to report the product_name, year, and price for each sale_id in the Sales table.

♥ **KEY INSIGHT:** Standard INNER JOIN linking transaction log rows with product dimension catalog tables.

INPUT: SALES

sale_id	product_id	year	quantity	price
1	100	2008	10	5000
2	100	2009	12	5000
7	200	2011	15	9000

INPUT: PRODUCT

product_id	product_name
100	Nokia
200	Apple
300	Samsung

EXPECTED OUTPUT

product_name	year	price
Nokia	2008	5000
Nokia	2009	5000
Apple	2011	9000

```
SELECT p.product_name, s.year, s.price
FROM Sales s
LEFT JOIN Product p
  ON s.product_id = p.product_id;
```

Q3 Customer Who Visited but Did Not Make Any Transactions

EASY

Write a solution to find the IDs of the users who visited without making transactions and the number of times they made these visits.

♥ **KEY INSIGHT:** Anti-join pattern: LEFT JOIN with WHERE right_table.key IS NULL isolates unconsummated visit events.

INPUT: VISITS

visit_id	customer_id
1	23
2	9
4	30
5	54
6	96
7	54
8	54

INPUT: TRANSACTIONS

transaction_id	visit_id	amount
2	5	310
3	5	300
9	5	200
12	1	910
13	2	970

EXPECTED OUTPUT

customer_id	count_no_trans
30	1
96	1
54	2

```
SELECT v.customer_id, COUNT(v.visit_id) AS count_no_trans
FROM Visits v
LEFT JOIN Transactions t
  ON v.visit_id = t.visit_id
WHERE t.transaction_id IS NULL
GROUP BY v.customer_id;
```

Q4 Rising Temperature

EASY

Write a solution to find all dates' id with higher temperatures compared to its previous dates (yesterday).

♥ **KEY INSIGHT:** Self-join table on date delta: `DATEDIFF(w1.recordDate, w2.recordDate) = 1` connects consecutive day entries.

INPUT: WEATHER

id	recordDate	temperature
1	2015-01-01	10
2	2015-01-02	25
3	2015-01-03	20
4	2015-01-04	30

EXPECTED OUTPUT

id
2
4

```
SELECT w2.id
FROM Weather w1
JOIN Weather w2
  ON DATEDIFF(w2.recordDate, w1.recordDate) = 1
WHERE w2.temperature > w1.temperature;

-- Alternative solution
SELECT id
FROM (
  SELECT
    id,
    temperature,
    LAG(temperature) OVER (ORDER BY recordDate) AS previous_temp
  FROM Weather
) t
WHERE temperature > previous_temp;
```

Q5 Average Time of Process per Machine**EASY**

There is a factory website that has several machines each running the same number of processes. Write a solution to find the average time each machine takes to complete a process rounded to 3 decimal places.

♥ **KEY INSIGHT:** Pair 'start' and 'end' records for the exact same process via multi-key self-join (machine_id + process_id).

INPUT: ACTIVITY

machine_id	process_id	activity_type	timestamp
0	0	start	0.712
0	0	end	1.520
0	1	start	3.140
0	1	end	4.120
1	0	start	0.550
1	0	end	1.550
1	1	start	0.430
1	1	end	1.420
2	0	start	4.100
2	0	end	4.512
2	1	start	2.500
2	1	end	5.000

EXPECTED OUTPUT

machine_id	processing_time
0	0.894
1	0.995
2	1.456

```
SELECT
    a1.machine_id,
    ROUND(AVG(a1.timestamp - a2.timestamp), 3) AS processing_time
FROM Activity a1
JOIN Activity a2
    ON a1.machine_id = a2.machine_id
    AND a1.process_id = a2.process_id
WHERE a1.activity_type = 'end'
    AND a2.activity_type = 'start'
GROUP BY a1.machine_id;
```

Q6 Employee Bonus**EASY**

Write a solution to report the name and bonus amount of each employee with a bonus less than 1000 or no bonus at all.

♥ **KEY INSIGHT:** Combine LEFT JOIN with a predicate that checks b.bonus < 1000 OR b.bonus IS NULL to include bonus-less staff.

INPUT: EMPLOYEE

empId	name	supervisor	salary
3	Brad	null	4000
1	John	3	1000
2	Dan	3	2000
4	Thomas	3	4000

INPUT: BONUS

empId	bonus
2	500
4	2000

EXPECTED OUTPUT

name	bonus
Brad	null
John	null
Dan	500

```
SELECT e.name AS name, b.bonus AS bonus
FROM Employee e
LEFT JOIN Bonus b
  ON e.empId = b.empId
WHERE b.bonus < 1000 OR b.bonus IS NULL;
```

Q7 Students and Examinations**EASY**

Write a solution to find the number of times each student attended each exam. Return the result table ordered by student_id and subject_name.

♥ **KEY INSIGHT:** CROSS JOIN guarantees every student-subject pair exists; LEFT JOIN onto attendance records counts zero attendances cleanly.

INPUT: STUDENTS

student_id	student_name
1	Alice
2	Bob
13	John
6	Alex

INPUT: EXAMINATIONS

student_id	subject_name
1	Math
1	Physics
1	Programming
2	Programming
1	Physics
1	Math
13	Math
13	Programming
13	Physics
2	Math
1	Math

INPUT: SUBJECTS

subject_name
Math
Physics
Programming

EXPECTED OUTPUT

student_id	student_name	subject_name	attended_exams
1	Alice	Math	3
1	Alice	Physics	2
1	Alice	Programming	1
2	Bob	Math	1
2	Bob	Physics	0
2	Bob	Programming	1
6	Alex	Math	0
6	Alex	Physics	0
6	Alex	Programming	0
13	John	Math	1
13	John	Physics	1
13	John	Programming	1

```
SELECT
    s.student_id,
    s.student_name,
    sub.subject_name,
    COUNT(e.subject_name) AS attended_exams
FROM Students s
CROSS JOIN Subjects sub
LEFT JOIN Examinations e
    ON s.student_id = e.student_id
    AND sub.subject_name = e.subject_name
GROUP BY
    s.student_id,
    s.student_name,
    sub.subject_name
ORDER BY
    s.student_id,
    sub.subject_name;
```

Q8 Managers with at Least 5 Direct Reports**MEDIUM**

Write a solution to find managers with at least five direct reports. Return the result table in any order.

♥ **KEY INSIGHT:** Self-join Employee as direct reports (e) to Manager (m), grouping by m.id and applying `HAVING COUNT(e.id) >= 5`.

INPUT: EMPLOYEE

id	name	department	managerId
101	John	A	null
102	Dan	A	101
103	James	A	101
104	Amy	A	101
105	Anne	A	101
106	Ron	B	101

EXPECTED OUTPUT

name
John

```
SELECT m.name
FROM employee AS e
JOIN employee AS m
      ON e.managerId = m.id
GROUP BY e.managerId
HAVING COUNT(e.id) >= 5;
```

Q9 Confirmation Rate**MEDIUM**

The confirmation rate of a user is the number of 'confirmed' messages divided by the total number of requested confirmation messages. Find the confirmation rate of each user, rounded to 2 decimal places.

♥ **KEY INSIGHT:** `AVG(c.action = 'confirmed')` evaluates boolean matches to 1 or 0, yielding the exact ratio automatically. Wrap in `IFNULL(..., 0)`.

INPUT: SIGNUPS

user_id	time_stamp
3	2020-03-21 10:16:13
7	2020-01-04 13:57:59
2	2020-07-29 23:09:44
6	2020-12-09 10:39:37

INPUT: CONFIRMATIONS

user_id	time_stamp	action
3	2021-01-06 03:30:46	timeout
3	2021-07-14 14:00:00	timeout
7	2021-06-12 11:57:29	confirmed
7	2021-06-13 12:58:28	confirmed
7	2021-06-14 13:59:27	confirmed
2	2021-01-22 00:00:00	confirmed
2	2021-02-28 23:59:59	timeout

EXPECTED OUTPUT

user_id	confirmation_rate
6	0.00
3	0.00
7	1.00
2	0.50

```
SELECT
    s.user_id,
    ROUND(
        IFNULL(
            COUNT(IF(c.action = 'confirmed', c.user_id, NULL))
            / COUNT(c.user_id),
            0
        ),
        2
    ) AS confirmation_rate
FROM Signups s
LEFT JOIN Confirmations c
    ON s.user_id = c.user_id
GROUP BY s.user_id;

-- Alternative solution
SELECT
    s.user_id,
    ROUND(AVG(IF(c.action = 'confirmed', 1, 0)), 2) AS confirmation_rate
FROM Signups AS s
LEFT JOIN Confirmations AS c
    ON s.user_id = c.user_id
GROUP BY user_id;
```

• Topic 3: Basic Aggregate Functions

Summarization functions, conditional aggregations, percentage fractions, and rolling KPIs.

Q1 Not Boring Movies

EASY

Write a solution to report the movies with an odd-numbered ID and a description that is not "boring". Return the result table ordered by rating in descending order.

♥ **KEY INSIGHT:** Modulo operator ($\text{id} \% 2 \neq 0$) filters odd keys; simple ORDER BY rating DESC performs numeric sorting.

INPUT: CINEMA

id	movie	description	rating
1	War	great 3D	8.9
2	Science	fiction	8.5
3	irish	boring	6.2
4	Ice song	Fantasy	8.6
5	House card	Interesting	9.1

EXPECTED OUTPUT

id	movie	description	rating
5	House card	Interesting	9.1
1	War	great 3D	8.9

```
# Write your MySQL query statement below
SELECT id, movie, description, rating
FROM Cinema
WHERE id % 2 != 0 AND description != 'boring'
ORDER BY rating DESC;
```

Q2 Average Selling Price

EASY

Write a solution to find the average selling price for each product. average_price should be rounded to 2 decimal places. If a product does not have any sold units, its average price is assumed to be 0.

♥ **KEY INSIGHT:** Weighted average formula $\sum(\text{price} \times \text{units}) / \sum(\text{units})$ joined on temporal validity interval (BETWEEN start_date AND end_date).

INPUT: PRICES

product_id	start_date	end_date	price
1	2019-02-17	2019-02-28	5
1	2019-03-01	2019-03-22	20
2	2019-02-01	2019-02-20	15
2	2019-02-21	2019-03-31	30

INPUT: UNITSSOLD

product_id	purchase_date	units
1	2019-02-25	100
1	2019-03-01	15
2	2019-02-10	200
2	2019-03-22	30

EXPECTED OUTPUT

product_id	average_price
1	6.96
2	16.96

```
SELECT
    product_id,
    ROUND(IFNULL(SUM(g) / SUM(units), 0), 2) AS average_price
FROM (
    SELECT
        p.product_id,
        s.units,
        s.units * p.price AS g
    FROM Prices p
    LEFT JOIN UnitsSold s
        ON p.product_id = s.product_id
        AND s.purchase_date BETWEEN p.start_date AND p.end_date
) t
GROUP BY product_id;
```

Q3 Project Employees I

EASY

Write an SQL query that reports the average experience years of all the employees for each project, rounded to 2 digits. Return the result table in any order.

♥ **KEY INSIGHT:** Standard aggregation with `AVG(e.experience_years)` grouped by the project primary key.

INPUT: PROJECT

project_id	employee_id
1	1
1	2
1	3
2	1
2	4

INPUT: EMPLOYEE

employee_id	name	experience_years
1	Khaled	3
2	Ali	2
3	John	1
4	Doe	2

EXPECTED OUTPUT

project_id	average_years
1	2.00
2	2.50

```
SELECT
    project_id,
    ROUND(IFNULL(SUM(total_experience) / COUNT(employee_id), 0), 2) AS average_years
FROM (
    SELECT
        p.project_id,
        p.employee_id,
        e.experience_years AS total_experience
    FROM Project p
    LEFT JOIN Employee e
        ON p.employee_id = e.employee_id
) t
GROUP BY project_id;
```

Q4 Percentage of Users Attended a Contest

EASY

Write a solution to find the percentage of the users registered in each contest rounded to two decimals. Return the result table ordered by percentage in descending order, then by contest_id ascending.

♥ **KEY INSIGHT:** Scalar subquery (SELECT COUNT(*) FROM Users) calculates total denominator dynamically.

INPUT: USERS (TOTAL = 3)

user_id	user_name
6	Alice
2	Bob
7	Alex

INPUT: REGISTER

contest_id	user_id
215	6
209	2
208	2
210	6
208	6
209	7
209	6
215	7
208	7
210	2
207	2
210	7

EXPECTED OUTPUT

contest_id	percentage
208	100.00
209	100.00
210	100.00
215	66.67
207	33.33

```
SELECT
    r.contest_id,
    ROUND(COUNT(r.user_id) * 100.0 / (SELECT COUNT(*) FROM Users), 2) AS percentage
FROM Register r
GROUP BY r.contest_id
ORDER BY percentage DESC, r.contest_id;
```

Q5 Queries Quality and Percentage

EASY

We define query quality as the average of the ratio between query rating and its position. We also define `poor_query_percentage` as the percentage of all queries with rating less than 3. Round both to 2 decimal places.

♥ **KEY INSIGHT:** `AVG(rating < 3) * 100` yields the percentage of rows satisfying boolean condition without a verbose CASE statement.

INPUT: QUERIES

query_name	result	position	rating
Dog	Golden Retriever	1	5
Dog	German Shepherd	2	5
Dog	Mule	200	1
Cat	Shirazi	5	2
Cat	Siamese	3	3
Cat	Sphynx	7	4

EXPECTED OUTPUT

query_name	quality	poor_query_percentage
Dog	2.50	33.33
Cat	0.66	33.33

```
SELECT
    query_name,
    ROUND(AVG(rating / position), 2) AS quality,
    ROUND(100 * AVG(rating < 3), 2) AS poor_query_percentage
FROM Queries
GROUP BY query_name;
```

-- Alternative solution 1

```
SELECT
    query_name,
    ROUND(AVG(rating / position), 2) AS quality,
    ROUND(
        SUM(CASE WHEN rating < 3 THEN 1 ELSE 0 END)
        * 100.0 / COUNT(*),
        2
    ) AS poor_query_percentage
FROM Queries
GROUP BY query_name;
```

-- Alternative solution 2

```
SELECT
    query_name,
    ROUND(AVG(rating / position), 2) AS quality,
    ROUND(
        COUNT(IF(rating < 3, 1, NULL)) * 100.0 / COUNT(*),
        2
    ) AS poor_query_percentage
FROM Queries
GROUP BY query_name;
```

Q6 Monthly Transactions I**MEDIUM**

Write an SQL query to find for each month and country, the number of transactions and their total amount, the number of approved transactions and their total amount.

♥ **KEY INSIGHT:** `DATE_FORMAT(trans_date, '%Y-%m')` partitions temporal data; conditional sum `SUM(CASE WHEN state = 'approved' THEN amount ELSE 0 END)` isolates approved balances.

INPUT: TRANSACTIONS

id	country	state	amount	trans_date
121	US	approved	1000	2018-12-18
122	US	declined	2000	2018-12-19
123	US	approved	2000	2019-01-01
124	DE	approved	2000	2019-01-07

EXPECTED OUTPUT

month	country	trans_count	approved_count	trans_total_amount	approved_total_amount
2018-12	US	2	1	3000	1000
2019-01	US	1	1	2000	2000
2019-01	DE	1	1	2000	2000

SELECT

```
DATE_FORMAT(trans_date, '%Y-%m') AS month,  
country,  
COUNT(id) AS trans_count,  
COUNT(IF(state = 'approved', id, NULL)) AS approved_count,  
SUM(amount) AS trans_total_amount,  
SUM(IF(state = 'approved', amount, 0)) AS approved_total_amount
```

```
FROM Transactions
```

```
GROUP BY month, country;
```

Q7 Immediate Food Delivery II**MEDIUM**

If the customer's preferred delivery date is the same as the order date, then the order is called immediate; otherwise, it is called scheduled. Find the percentage of immediate orders in the first orders of all customers, rounded to 2 decimal places.

♥ **KEY INSIGHT:** Tuple subquery (customer_id, order_date) IN (SELECT customer_id, MIN(order_date) ...) isolates initial customer transactions.

INPUT: DELIVERY

delivery_id	customer_id	order_date	customer_pref_delivery_date
1	1	2019-08-01	2019-08-02
2	2	2019-08-02	2019-08-02
3	1	2019-08-11	2019-08-11
4	3	2019-08-24	2019-08-24
5	3	2019-08-21	2019-08-22
6	2	2019-08-11	2019-08-13
7	4	2019-08-09	2019-08-09

EXPECTED OUTPUT

immediate_percentage
50.00

```
SELECT
    ROUND(AVG(order_date = customer_pref_delivery_date) * 100, 2) AS immediate_percentage
FROM Delivery
WHERE (customer_id, order_date) IN (
    SELECT customer_id, MIN(order_date)
    FROM Delivery
    GROUP BY customer_id
);

-- Alternative solution
SELECT
    ROUND(
        COUNT(IF(order_date = customer_pref_delivery_date, 1, NULL))
        * 100.0 / COUNT(*),
        2
    ) AS immediate_percentage
FROM Delivery
WHERE (customer_id, order_date) IN (
    SELECT customer_id, MIN(order_date)
    FROM Delivery
    GROUP BY customer_id
);
```

Q8 Game Play Analysis IV**MEDIUM**

Write a solution to report the fraction of players that logged in again on the day after the day they first logged in, rounded to 2 decimal places.

♥ **KEY INSIGHT:** Compute initial login dates via subquery, then LEFT JOIN to check if another event occurred exactly +1 day later.

INPUT: ACTIVITY

player_id	device_id	event_date	games_played
1	2	2016-03-01	5
1	2	2016-03-02	6
2	3	2017-06-25	1
3	1	2016-03-02	0
3	4	2018-07-03	5

EXPECTED OUTPUT

fraction
0.33

```
SELECT ROUND(
    COUNT(*) / (SELECT COUNT(DISTINCT player_id) FROM Activity),
    2
) AS fraction
FROM Activity a
WHERE (a.player_id, a.event_date) IN (
    SELECT player_id, DATE_ADD(MIN(event_date), INTERVAL 1 DAY)
    FROM Activity
    GROUP BY player_id
);
```

● Topic 4: Sorting and Grouping

Grouping criteria, relational division, count distinct metrics, and filtering groups using HAVING.

Q1 Number of Unique Subjects Taught by Each Teacher

EASY

Write a solution to calculate the number of unique subjects each teacher teaches in the university. Return the result table in any order.

♥ **KEY INSIGHT:** COUNT(DISTINCT subject_id) ignores duplicate offerings of the same subject across different departments.

INPUT: TEACHER

teacher_id	subject_id	dept_id
1	2	3
1	2	4
1	3	3
2	1	1
2	2	1
2	3	1
2	4	1

EXPECTED OUTPUT

teacher_id	cnt
1	2
2	4

Write your MySQL query statement below

```
SELECT teacher_id, COUNT(DISTINCT subject_id) AS cnt
FROM Teacher
GROUP BY teacher_id;
```

Q2 User Activity for the Past 30 Days I**EASY**

Write a solution to find the daily active user count for a period of 30 days ending 2019-07-27 inclusively. A user was active on days they made at least one activity.

♥ **KEY INSIGHT:** Inclusive date window filtering using BETWEEN '2019-06-28' AND '2019-07-27' with distinct user counts.

INPUT: ACTIVITY

user_id	session_id	activity_date	activity_type
1	1	2019-07-20	open_session
1	1	2019-07-20	scroll_down
1	1	2019-07-20	end_session
2	4	2019-07-20	open_session
2	4	2019-07-21	send_message
2	4	2019-07-21	end_session
3	2	2019-07-21	open_session
3	2	2019-07-21	send_message
3	2	2019-07-21	end_session
4	3	2019-06-25	open_session

EXPECTED OUTPUT

day	active_users
2019-07-20	2
2019-07-21	2

```
SELECT activity_date AS day, COUNT(DISTINCT user_id) AS active_users
FROM Activity
WHERE (activity_date > '2019-06-27' AND activity_date <= '2019-07-27')
GROUP BY activity_date;
```

Q3 Product Sales Analysis III**MEDIUM**

Write a solution to select the product_id, year, quantity, and price for the first year of every product sold.

♥ **KEY INSIGHT:** Identify each product's initial year using (product_id, MIN(year)) in a correlated/in subquery condition.

INPUT: SALES

sale_id	product_id	year	quantity	price
1	100	2008	10	5000
2	100	2009	12	5000
7	200	2011	15	9000

EXPECTED OUTPUT

product_id	first_year	quantity	price
100	2008	10	5000
200	2011	15	9000

```
SELECT product_id, year AS first_year, quantity, price
FROM Sales
WHERE (product_id, year) IN (
    SELECT product_id, MIN(year)
    FROM Sales
    GROUP BY product_id
);
```

Q4 Classes More Than 5 Students

EASY

Write a solution to find all the classes that have at least five students. Return the result table in any order.

♥ **KEY INSIGHT:** GROUP BY class combined with post-aggregation filtering HAVING COUNT(student) >= 5.

INPUT: COURSES

student	class
A	Math
B	English
C	Math
D	Biology
E	Math
F	Computer
G	Math
H	Math
I	Math

EXPECTED OUTPUT

class
Math

```
SELECT class
FROM Courses
GROUP BY class
HAVING COUNT(student) >= 5;
```

Q5 Find Followers Count

EASY

Write a solution that will, for each user, return the number of followers. Return the result table ordered by user_id in ascending order.

♥ **KEY INSIGHT:** Simple aggregation with COUNT(follower_id) ordered by the grouping key.

INPUT: FOLLOWERS

user_id	follower_id
0	1
1	0
2	0
2	1

EXPECTED OUTPUT

user_id	followers_count
0	1
1	1
2	2

```
# Write your MySQL query statement below
SELECT user_id, COUNT(follower_id) AS followers_count
FROM Followers
GROUP BY user_id
ORDER BY user_id;
```

Q6 Biggest Single Number**EASY**

A single number is a number that appeared only once in the MyNumbers table. Find the largest single number. If there is no single number, report null.

♥ **KEY INSIGHT:** Wrap the `HAVING COUNT(num) = 1` subquery inside an outer `MAX()`; if no rows qualify, `MAX()` gracefully evaluates to `NULL`.

INPUT: MYNUMBERS

num
8
8
3
3
1
4
5
6

EXPECTED OUTPUT

num
6

Write your MySQL query statement below

```
SELECT MAX(num) AS num
FROM (
  SELECT num
  FROM MyNumbers
  GROUP BY num
  HAVING COUNT(num) = 1
) t;
```

Q7 Customers Who Bought All Products**MEDIUM**

Write a solution to report the customer ids from the Customer table who bought all the products in the Product table.

♥ **KEY INSIGHT:** Relational division: Match distinct purchased count `COUNT(DISTINCT product_key)` against full catalog count `(SELECT COUNT(*) FROM Product)`.

INPUT: CUSTOMER

customer_id	product_key
1	5
2	6
3	5
3	6
1	6

INPUT: PRODUCT

product_key
5
6

EXPECTED OUTPUT

customer_id
1
3

```
SELECT customer_id
FROM Customer
GROUP BY customer_id
HAVING COUNT(DISTINCT product_key) = (SELECT COUNT(product_key) FROM Product);
```

● Topic 5: Advanced Select & Joins

Consecutive sequence detection, cumulative window frames, conditional categorization, and UNION queries.

Q1 The Number of Employees Which Report to Each Employee

EASY

For this problem, we will consider a manager an employee who has at least 1 other employee reporting to them. Write a solution to report the employee_id, name, reports_count, and average_age of all managers, rounded to the nearest integer.

♥ **KEY INSIGHT:** Self-join linking manager m.employee_id to reportee e.reports_to, applying ROUND(AVG(e.age)).

INPUT: EMPLOYEES

employee_id	name	reports_to	age
9	Hercy	null	43
6	Alice	9	41
4	Bob	9	36
2	Winston	null	37

EXPECTED OUTPUT

employee_id	name	reports_count	average_age
9	Hercy	2	39

```
SELECT
    m.employee_id,
    m.name,
    COUNT(e.employee_id) AS reports_count,
    ROUND(AVG(e.age)) AS average_age
FROM employees e
JOIN employees m
    ON e.reports_to = m.employee_id
GROUP BY m.employee_id, m.name
ORDER BY m.employee_id;
```

Q2 Primary Department for Each Employee

EASY

Employees can belong to multiple departments. When an employee joins other departments, they need to decide which department is their primary department (primary_flag = 'Y'). If an employee belongs to only one department, their primary flag is 'N'. Report all employees with their primary department.

♥ **KEY INSIGHT:** Select employees where primary_flag = 'Y' OR whose total associated department count is strictly 1.

INPUT: EMPLOYEE

employee_id	department_id	primary_flag
1	1	N
2	1	Y
2	2	N
3	3	N
4	2	N
4	3	Y
4	4	N

EXPECTED OUTPUT

employee_id	department_id
1	1
2	1
3	3
4	3

```
SELECT employee_id, department_id
FROM Employee
WHERE primary_flag = 'Y'
   OR employee_id IN (
    SELECT employee_id
    FROM Employee
    GROUP BY employee_id
    HAVING COUNT(department_id) = 1
);
```

Q3 Triangle Judgement

EASY

Report for every three line segments whether they can form a triangle (sum of any two sides must be strictly greater than the third side).

♥ **KEY INSIGHT:** Triangle inequality theorem verified via multi-clause CASE WHEN $x+y>z$ AND $x+z>y$ AND $y+z>x$ THEN 'Yes' ELSE 'No' END.

INPUT: TRIANGLE

x	y	z
13	15	30
10	20	15

EXPECTED OUTPUT

x	y	z	triangle
13	15	30	No
10	20	15	Yes

```
SELECT
  x,
  y,
  z,
  CASE
    WHEN x + y > z AND x + z > y AND y + z > x THEN 'Yes'
    ELSE 'No'
  END AS `triangle`
FROM Triangle;

-- Alternative solution
SELECT
  *,
  IF(x + y > z AND y + z > x AND x + z > y, 'Yes', 'No') AS triangle
FROM Triangle;
```

Q4 Consecutive Numbers**MEDIUM**

Find all numbers that appear at least three times consecutively in the Logs table. Return the result table in any order.

♥ **KEY INSIGHT:** LEAD(num, 1) and LEAD(num, 2) window functions look ahead 1 and 2 rows to evaluate consecutive triplicates.

INPUT: LOGS

id	num
1	1
2	1
3	1
4	2
5	1
6	2
7	2

EXPECTED OUTPUT

ConsecutiveNums
1

```
SELECT num AS ConsecutiveNums
FROM (
  SELECT
    num,
    LEAD(num) OVER () AS next,
    LAG(num) OVER () AS previous
  FROM Logs
) t
WHERE num = next AND num = previous
GROUP BY num;
```

Q5 Product Price at a Given Date**MEDIUM**

Write a solution to find the prices of all products on 2019-08-16. Assume the price of all products before any change is 10.

♥ **KEY INSIGHT:** UNION latest price update on/before cutoff with products whose initial price change happened strictly after cutoff (defaulting to 10).

INPUT: PRODUCTS

product_id	new_price	change_date
1	20	2019-08-14
2	50	2019-08-14
1	30	2019-08-15
1	35	2019-08-16
2	65	2019-08-17
3	20	2019-08-18

EXPECTED OUTPUT

product_id	price
1	35
2	50
3	10

```
SELECT product_id, new_price AS price
FROM Products
WHERE (product_id, change_date) IN (
    SELECT product_id, MAX(change_date)
    FROM Products
    WHERE change_date <= '2019-08-16'
    GROUP BY product_id
)
UNION
SELECT DISTINCT product_id, 10 AS price
FROM Products
WHERE product_id NOT IN (
    SELECT product_id
    FROM Products
    WHERE change_date <= '2019-08-16'
);

-- Alternative solution
SELECT
    p.product_id,
    COALESCE((
        SELECT pr.new_price
        FROM products pr
        WHERE p.product_id = pr.product_id
            AND pr.change_date <= '2019-08-16'
        ORDER BY pr.change_date DESC LIMIT 1
    ), 10) AS price
FROM (
    SELECT DISTINCT product_id
    FROM products
) AS p;
```

Q6 Last Person to Fit in the Bus**MEDIUM**

There is a queue of people waiting to board a bus. However, the bus has a weight limit of 1000 kilograms, so there may be some people who cannot board. Find the `person_name` of the last person that can fit into the bus without exceeding the weight limit.

♥ **KEY INSIGHT:** Cumulative window sum `SUM(weight) OVER (ORDER BY turn)` generates running total; filter ≤ 1000 and take top row ordered DESC.

INPUT: QUEUE

person_id	person_name	weight	turn
5	Alice	250	1
4	Bob	175	2
3	Alex	350	3
6	John Cena	400	4
1	Winston	500	5
2	Marie	200	6

EXPECTED OUTPUT

person_name
John Cena

```
WITH new_table AS (  
  SELECT  
    person_name,  
    SUM(weight) OVER (ORDER BY turn) AS cumulative_weight  
  FROM Queue  
)  
SELECT person_name  
FROM new_table  
WHERE cumulative_weight <= 1000  
ORDER BY cumulative_weight DESC  
LIMIT 1;
```

Q7 Count Salary Categories**MEDIUM**

Write a solution to calculate the number of bank accounts for each salary category. The salary categories are: "Low Salary" (strictly less than \$20,000), "Average Salary" (inclusive between \$20,000 and \$50,000), and "High Salary" (strictly greater than \$50,000). The result table must contain all three categories. If there are no accounts in a category, return 0.

♥ **KEY INSIGHT:** UNION static category queries to ensure categories with zero accounts appear in output with 0 rather than being omitted.

INPUT: ACCOUNTS

account_id	income
3	108939
2	12747
8	87709
6	91796

EXPECTED OUTPUT

category	accounts_count
Low Salary	1
Average Salary	0
High Salary	3

```
SELECT
    'Low Salary' AS category,
    COUNT(CASE WHEN income < 20000 THEN 1 END) AS accounts_count
FROM accounts
UNION ALL
SELECT
    'Average Salary' AS category,
    COUNT(CASE WHEN income BETWEEN 20000 AND 50000 THEN 1 END) AS accounts_count
FROM accounts
UNION ALL
SELECT
    'High Salary' AS category,
    COUNT(IF(income > 50000, 1, NULL)) AS accounts_count
FROM accounts;

-- Handy patterns
COUNT(CASE WHEN condition THEN 1 END)
SUM(CASE WHEN condition THEN 1 ELSE 0 END)
COUNT(IF(condition, 1, NULL))
SUM(IF(condition, 1, 0))
```

● Topic 6: Subqueries

Correlated subqueries, multi-row evaluations, dense ranking partitions, and dynamic date rolling windows.

Q1 Employees Whose Manager Left the Company

EASY

Find the IDs of the employees whose salary is strictly less than \$30,000 and whose manager left the company. When a manager leaves the company, their information is deleted from the Employees table, but their reports still have their manager_id set to the manager that left. Return the result table ordered by employee_id.

♥ **KEY INSIGHT:** manager_id NOT IN (SELECT employee_id FROM Employees) detects deleted managers; filter out manager_id IS NOT NULL first.

INPUT: EMPLOYEES

employee_id	name	manager_id	salary
3	Mila	9	60301
12	Anton	null	31000
13	Emery	null	67084
1	Kalel	11	21241
9	Mikaela	null	50937
11	Joziah	6	28485

EXPECTED OUTPUT

employee_id
11

```
SELECT e.employee_id
FROM Employees e
LEFT JOIN Employees m
  ON e.manager_id = m.employee_id
WHERE e.salary < 30000
  AND e.manager_id NOT IN (SELECT employee_id FROM Employees)
ORDER BY employee_id;
```

-- Alternative solution

```
SELECT employee_id
FROM Employees
WHERE salary < 30000
  AND manager_id NOT IN (
    SELECT employee_id FROM Employees
  )
ORDER BY employee_id;
```

Q2 Exchange Seats**MEDIUM**

Write a solution to swap the seat id of every two consecutive students. If the number of students is odd, the id of the last student is not swapped. Return the result table ordered by id in ascending order.

♥ **KEY INSIGHT:** Conditional math via CASE: odd numbers shift to id+1, evens shift to id-1, except the last odd record which remains id.

INPUT: SEAT

id	student
1	Abbot
2	Doris
3	Emerson
4	Green
5	Jeames

EXPECTED OUTPUT

id	student
1	Doris
2	Abbot
3	Green
4	Emerson
5	Jeames

```
SELECT
  id,
  CASE
    WHEN id % 2 = 1 AND id < (SELECT MAX(id) FROM Seat)
      THEN (SELECT student FROM Seat s2 WHERE s2.id = Seat.id + 1)
    WHEN id % 2 = 0
      THEN (SELECT student FROM Seat s2 WHERE s2.id = Seat.id - 1)
    ELSE student
  END AS student
FROM Seat
ORDER BY id;
```

Q3 Movie Rating**MEDIUM**

Write a solution to: (1) Find the name of the user who has rated the greatest number of movies (tie: lexicographically smaller name). (2) Find the movie name with the highest average rating in February 2020 (tie: lexicographically smaller title).

♥ **KEY INSIGHT:** Execute two independent subqueries with ORDER BY ... LIMIT 1 and stack their single-column result using UNION ALL.

INPUT: USERS

user_id	name
1	Daniel
2	Monica
3	Maria
4	James

INPUT: MOVIES

movie_id	title
1	Avengers
2	Frozen 2
3	Joker

INPUT: MOVIERATING

movie_id	user_id	rating	created_at
1	1	3	2020-01-12
1	2	4	2020-02-11
1	3	2	2020-02-12
1	4	1	2020-01-01
2	1	5	2020-02-17
2	2	2	2020-02-01
2	3	2	2020-03-01
3	1	3	2020-02-22
3	2	4	2020-02-25

EXPECTED OUTPUT

results
Daniel
Frozen 2

```
SELECT results
FROM (
    SELECT u.name AS results
    FROM Users u
    JOIN MovieRating mr
        ON u.user_id = mr.user_id
    GROUP BY u.user_id, u.name
    ORDER BY COUNT(mr.movie_id) DESC, u.name ASC
    LIMIT 1
) a
UNION ALL
SELECT results
FROM (
    SELECT m.title AS results
    FROM Movies m
    JOIN MovieRating mr
        ON m.movie_id = mr.movie_id
    WHERE mr.created_at >= '2020-02-01'
        AND mr.created_at < '2020-03-01'
    GROUP BY m.movie_id, m.title
    ORDER BY AVG(mr.rating) DESC, m.title ASC
    LIMIT 1
) b;
```

Q4 Restaurant Growth**MEDIUM**

Compute moving 7-day window average and sum of daily amounts paid by customers, rounded to 2 decimals. Return the result table ordered by visited_on in ascending order.

♥ **KEY INSIGHT:** Aggregate daily totals first in a CTE; self-join with DATEDIFF BETWEEN 0 AND 6 and filter HAVING COUNT = 7 for full 7-day windows.

INPUT: CUSTOMER

customer_id	name	visited_on	amount
1	Jhon	2019-01-01	100
2	Daniel	2019-01-02	110
3	Jade	2019-01-03	120
4	Khaled	2019-01-04	130
5	Winston	2019-01-05	110
6	Elvis	2019-01-06	140
7	Anna	2019-01-07	150
8	Maria	2019-01-08	80
9	Jaze	2019-01-09	110
1	Jhon	2019-01-10	130
3	Jade	2019-01-10	150

EXPECTED OUTPUT

visited_on	amount	average_amount
2019-01-07	860	122.86
2019-01-08	840	120.00
2019-01-09	840	120.00
2019-01-10	1000	142.86

```
SELECT visited_on, amount, average_amount
FROM (
    SELECT DISTINCT
        visited_on,
        SUM(amount) OVER (
            ORDER BY visited_on
            RANGE BETWEEN INTERVAL 6 DAY PRECEDING AND CURRENT ROW
        ) AS amount,
        ROUND(
            SUM(amount) OVER (
                ORDER BY visited_on
                RANGE BETWEEN INTERVAL 6 DAY PRECEDING AND CURRENT ROW
            ) / 7,
            2
        ) AS average_amount
    FROM Customer
) AS whole_totals
WHERE DATEDIFF(visited_on, (SELECT MIN(visited_on) FROM Customer)) >= 6;
```

Q5 Friend Requests II: Who Has the Most Friends**MEDIUM**

Write a solution to find the people who have the most friends and the most friends number. The test cases are generated so that only one person has the most friends.

♥ **KEY INSIGHT:** Bidirectional relationships: UNION ALL requester and acceptor columns into a unified user stream, then COUNT(*) GROUP BY id.

INPUT: REQUESTACCEPTED

requester_id	accepter_id	accept_date
1	2	2016/06/03
1	3	2016/06/08
2	3	2016/06/08
3	4	2016/06/09

EXPECTED OUTPUT

id	num
3	3

```
WITH all_ids AS (  
    SELECT requester_id AS id FROM RequestAccepted  
    UNION ALL  
    SELECT acceptor_id AS id FROM RequestAccepted  
)  
SELECT id, COUNT(id) AS num  
FROM all_ids  
GROUP BY id  
ORDER BY num DESC  
LIMIT 1;
```

```
# Write your MySQL query statement below  
-- select requester_id as id,  
-- (select count(*) from RequestAccepted  
-- where id=requester_id or id=acceptor_id) as num  
-- from RequestAccepted  
-- group by requester_id  
-- order by num desc limit 1
```

Q6 Investments in 2016**MEDIUM**

Write a solution to report the sum of all total investment values in 2016 `tiv_2016` for all policyholders who: (1) have the same `tiv_2015` value as one or more other policyholders, and (2) are not located in the same city as any other policyholder (unique `lat`, `lon`). Round `tiv_2016` to 2 decimal places.

♥ **KEY INSIGHT:** Two independent conditions: non-unique investment value (`HAVING COUNT(*) > 1`) AND strictly unique geospatial coordinates (`HAVING COUNT(*) = 1`).

INPUT: INSURANCE

pid	tiv_2015	tiv_2016	lat	lon
1	10	5	10	10
2	20	20	20	20
3	10	30	20	20
4	10	40	40	40

EXPECTED OUTPUT

tiv_2016
45.00

```
SELECT ROUND(SUM(tiv_2016), 2) AS tiv_2016
FROM Insurance
WHERE tiv_2015 IN (
    SELECT tiv_2015
    FROM Insurance
    GROUP BY tiv_2015
    HAVING COUNT(tiv_2015) > 1
)
AND (lat, lon) IN (
    SELECT lat, lon
    FROM Insurance
    GROUP BY lat, lon
    HAVING COUNT(*) = 1
);
```

Q7 Department Top Three Salaries**HARD**

A company's executives are interested in seeing who earns the most money in each of the company's departments. A high earner in a department is an employee who has a salary in the top three unique salaries for that department. Write a solution to find the employees who are high earners in each of the departments.

♥ **KEY INSIGHT:** `DENSE_RANK() OVER (PARTITION BY departmentId ORDER BY salary DESC)` ensures ties do not skip ranks, correctly identifying top 3 unique compensation levels.

INPUT: EMPLOYEE

id	name	salary	departmentId
1	Joe	85000	1
2	Henry	80000	2
3	Sam	60000	2
4	Max	90000	1
5	Janet	69000	1
6	Randy	85000	1
7	Will	70000	1

INPUT: DEPARTMENT

id	name
1	IT
2	Sales

EXPECTED OUTPUT

Department	Employee	Salary
IT	Max	90000
IT	Joe	85000
IT	Randy	85000
IT	Will	70000
Sales	Henry	80000
Sales	Sam	60000

```
SELECT
    d.name AS Department,
    t.name AS Employee,
    t.salary AS Salary
FROM (
    SELECT
        e.departmentId,
        e.name,
        e.salary,
        DENSE_RANK() OVER (PARTITION BY e.departmentId ORDER BY salary DESC) AS `DR`
    FROM Employee e
) t
JOIN Department d
    ON t.departmentId = d.id
WHERE t.DR <= 3;
```

● Topic 7: Advanced String Functions / Regex / Clause

String manipulation, custom capitalization, pattern matching with regular expressions, and group concatenation.

Q1 Fix Names in a Table

EASY

Write a solution to fix the names so that only the first character is uppercase and the rest are lowercase. Return the result table ordered by user_id.

♥ **KEY INSIGHT:** `CONCAT(UPPER(SUBSTRING(name, 1, 1)), LOWER(SUBSTRING(name, 2)))` formats title casing in standard SQL.

INPUT: USERS

user_id	name
1	aLice
2	bOB

EXPECTED OUTPUT

user_id	name
1	Alice
2	Bob

```
SELECT
    user_id,
    CONCAT(UPPER(LEFT(name, 1)), LOWER(SUBSTRING(name, 2))) AS name
FROM Users
ORDER BY user_id;
```

Q2 Patients With a Condition

EASY

Write a solution to find the patient_id, patient_name, and conditions of the patients who have Type I Diabetes. Type I Diabetes always starts with DIAB1 prefix.

♥ **KEY INSIGHT:** Check both prefix at start of string (`LIKE 'DIAB1%'`) and prefix within space-separated tokens (`LIKE '% DIAB1%'`).

INPUT: PATIENTS

patient_id	patient_name	conditions
1	Daniel	YFEV COUGH
2	Alice	
3	Bob	DIAB100 MYOP
4	George	ACNE DIAB100
5	Alain	DIAB201

EXPECTED OUTPUT

patient_id	patient_name	conditions
3	Bob	DIAB100 MYOP
4	George	ACNE DIAB100

```
SELECT patient_id, patient_name, conditions
FROM Patients
WHERE conditions LIKE 'DIAB1%'
      OR conditions LIKE '% DIAB1%';
```

Q3 Delete Duplicate Emails

EASY

Write a solution to delete all duplicate emails, keeping only one unique email with the smallest id. Note that you are supposed to write a DELETE statement and not a SELECT one.

♥ **KEY INSIGHT:** Self-join on matching emails where $p1.id > p2.id$ allows direct deletion of larger redundant ID rows.

INPUT: PERSON

id	email
1	john@example.com
2	bob@example.com
3	john@example.com

EXPECTED OUTPUT

id	email
1	john@example.com
2	bob@example.com

```
DELETE p1
FROM Person p1
JOIN Person p2
  ON p1.email = p2.email
  AND p1.id > p2.id;
```

Q4 Second Highest Salary

MEDIUM

Write a solution to find the second highest distinct salary from the Employee table. If there is no second highest salary, return null.

♥ **KEY INSIGHT:** Wrapping `LIMIT 1 OFFSET 1` in an outer scalar `SELECT (...)` forces a NULL return if only 1 distinct salary exists.

INPUT: EMPLOYEE

id	salary
1	100
2	200
3	300

EXPECTED OUTPUT

SecondHighestSalary
200

```
SELECT MAX(salary) AS SecondHighestSalary
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);
```

Q5 Group Sold Products By The Date**EASY**

Write a solution to find for each date the number of different products sold and their names. The sold products names for each date should be sorted lexicographically. Return the result table ordered by `sell_date`.

♥ **KEY INSIGHT:** `GROUP_CONCAT(DISTINCT product ORDER BY product SEPARATOR ',')` aggregates grouped values into a sorted CSV list.

INPUT: ACTIVITIES

sell_date	product
2020-05-30	Headphone
2020-06-01	Pencil
2020-06-02	Mask
2020-05-30	Basketball
2020-06-01	Bible
2020-05-30	T-Shirt

EXPECTED OUTPUT

sell_date	num_sold	products
2020-05-30	3	Basketball,Headphone,T-Shirt
2020-06-01	2	Bible,Pencil
2020-06-02	1	Mask

SELECT

```
sell_date,  
COUNT(DISTINCT product) AS num_sold,  
GROUP_CONCAT(DISTINCT product ORDER BY product ASC SEPARATOR ',') AS products
```

```
FROM Activities
```

```
GROUP BY sell_date
```

```
ORDER BY sell_date ASC;
```

Q6 List the Products Ordered in a Period**EASY**

Write a solution to get the names of products that have at least 100 units ordered in February 2020 and their amount. Return the result table in any order.

♥ **KEY INSIGHT:** Filter month via `DATE_FORMAT = '2020-02'` before grouping, then filter cumulative volume via `HAVING SUM(o.unit) >= 100`.

INPUT: ORDERS

product_id	order_date	unit
1	2020-02-05	60
1	2020-02-10	70
2	2020-01-18	30
2	2020-02-11	80
3	2020-02-17	2
3	2020-02-24	3
4	2020-03-01	20
4	2020-03-04	30
4	2020-03-04	60
5	2020-02-25	50
5	2020-02-27	50
5	2020-03-01	50

INPUT: PRODUCTS

product_id	product_name	product_category
1	Leetcode Solutions	Book
2	Jewels of Stringology	Book
3	HP	Laptop
4	Lenovo	Laptop
5	Leetcode Kit	T-shirt

EXPECTED OUTPUT

product_name	unit
Leetcode Solutions	130
Leetcode Kit	100

```
SELECT p.product_name, t.unit
FROM Products p
JOIN (
    SELECT product_id, SUM(unit) AS unit
    FROM Orders
    WHERE DATE_FORMAT(order_date, '%Y-%m') = '2020-02'
    GROUP BY product_id
) t
ON p.product_id = t.product_id
WHERE t.unit >= 100;

-- Alternative solution 1
SELECT p.product_name, t.unit
FROM Products p,
(
    SELECT product_id, SUM(unit) AS unit
    FROM Orders
    WHERE MONTH(order_date) = 2
    GROUP BY product_id
) t
WHERE p.product_id = t.product_id
AND t.unit >= 100;

-- Alternative solution 2
SELECT p.product_name, SUM(o.unit) AS unit
FROM Products p
JOIN Orders o
ON o.product_id = p.product_id
WHERE o.order_date LIKE '2020-02-%'
GROUP BY o.product_id
HAVING unit >= 100;
```

Q7 Find Users With Valid E-Mails

EASY

Write a solution to find the users who have valid emails. A valid e-mail has a prefix name and a domain: (1) The prefix name is a string that may contain letters (upper or lower case), digits, underscore '_', period '.', and/or dash '-'. The prefix name must start with a letter. (2) The domain is '@leetcode.com'.

♥ **KEY INSIGHT:** Regex anchors: `^[A-Za-z]` enforces initial letter prefix, `[A-Za-z0-9_.-]*` allows valid characters, and `@leetcode[.]com$` pins domain.

INPUT: USERS

user_id	name	mail
1	Winston	winston@leetcode.com
2	Jonathan	jonathanisgreat
3	Annabelle	bella-@leetcode.com
4	Sally	sally.come@leetcode.com
5	Marwan	quarz#2020@leetcode.com
6	David	david69@gmail.com
7	Shapiro	.shapo@leetcode.com

EXPECTED OUTPUT

user_id	name	mail
1	Winston	winston@leetcode.com
3	Annabelle	bella-@leetcode.com
4	Sally	sally.come@leetcode.com

Write your MySQL query statement below

```
SELECT *  
FROM Users  
WHERE REGEXP_LIKE(mail, '^[A-Za-z][A-Za-z0-9_.-]*@leetcode\\.com$', 'c');
```